Summer Internship Project Report

on

AI-BASED AUTONOMOUS VEHICLE FOR SMART CAMPUS NAVIGATION

Submitted by

In partial fulfillment of the requirements for the completion of

INTERNSHIP PROGRAM

By

RAPARTHI ARUN	(23641A05G7)
KOTA ANUJ KUMAR	(23641A05H7)
THANGELLAPELLY KARTHIK	(23641A05L4)
DOMAKUNTLA LAHARI	(24641A05H0)

Vaagdevi college of enginnerring,bollikunta Warangal

INTERNSHIP DURATION

(May 05, 2026 – July 05, 2026)

Under the supervision of

Prof L. Anjaneyulu
Professor, Dept of ECE
Head, Centre for Innovation and Incubation

NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

INTERNSHIP CERTIFICATE

This is to certify that the Project work entitled **"AI-Based Autonomous Vehicle For Smart Campus Navigation"** is a bonafide record of work carried out by Raparthi Arun (23641a05G7),Kota Anuj kumar(23641a05H7.), Thangellapelly Karthik(23641a05L4.),Domakuntla Lahari (24641A05H0),Vaagdevi college of engineering, Warangal at **Center for Innovation and Incubation, NIT, Warangal** from May 05-2026 to July 05-2026 as part of Summer Internship.

Prof. L. Anjaneyulu

Head, Center for Innovation and Incubation

ACKNOWLEDGEMENT

I would like to express my sincere gratitude and heartfelt thanks to **Dr. L. Anjaneyulu, Professor, Department of ECE, Chairman of the Institute of Engineers (Warangal Local Centre), Head of the Centre for Innovation and Incubation, and IPR Cell at National Institute of Technology Warangal,** for providing us with this valuable opportunity and continuous guidance throughout our learning journey.

His inspiring mentorship, technical expertise, and constant encouragement motivated us to explore innovative ideas and gain practical knowledge in a professional research environment. We are truly thankful for the support, valuable suggestions, and exposure to advanced laboratories and the innovative ecosystem at NIT Warangal.

We also extend our deep appreciation for his dedication towards nurturing students and encouraging innovation, research, and real-world problem-solving skills. Working under his guidance has been a great learning experience and a memorable opportunity for us.

We sincerely thank him for his time, support, and encourage

I would like to express my sincere gratitude and heartfelt thanks to **Mrs. P. Shailaja, Associate Professor, Department of Computer Science and Engineering, Vaagdevi College of Engineering**, for her valuable guidance, continuous encouragement, and unwavering support throughout the development of this project.

Her expertise, constructive suggestions, and timely assistance greatly contributed to the successful completion of this work. I am deeply thankful for the time and effort she invested in helping us understand the project requirements and guiding us at every stage. I extend my sincere appreciation for her motivation, patience, and dedication, which inspired us to overcome challenges and achieve our project objectives successfully.

I would also like to acknowledge her valuable support in coordinating project-related activities and providing the necessary guidance required for the preparation and completion of this work.

ABSTRACT

The main objective of our Autonomous Vehicle project is to design and develop an intelligent vehicle capable of traveling from one point to another while automatically detecting and avoiding obstacles in real time. The system combines Artificial Intelligence (AI), Machine Learning (ML), Computer Vision, and embedded systems to achieve smart and efficient navigation.

The vehicle uses ultrasonic sensors and cameras to continuously monitor the surrounding environment. YOLO (You Only Look Once) object detection algorithms are implemented for detecting vehicles, pedestrians, obstacles, and vehicle number plates. The detected information is processed using intelligent algorithms to make navigation decisions and avoid collisions safely.

The project also integrates Arduino, motor drivers, and sensor modules for controlling vehicle movement and direction. The autonomous system is designed to improve transportation safety, reduce human intervention, and enhance driving efficiency through automation and real-time decision-making.

This project provides hands-on experience in robotics, AI, computer vision, and autonomous navigation technologies. It demonstrates the practical implementation of smart transportation systems and highlights the future potential of autonomous vehicles in real-world applications

INTRODUCTION

Autonomous vehicles are an advanced application of Artificial Intelligence (AI), robotics, and embedded systems in modern transportation. This project focuses on developing a smart autonomous robotic vehicle capable of destination reaching, road monitoring, traffic surveillance, and obstacle avoidance. The system uses intelligent algorithms, sensors, and real-time data processing for autonomous navigation. The vehicle continuously monitors its surroundings and makes decisions to ensure safe movement. Raspberry Pi serves as the central processing unit for AI operations, image processing, and web communication. Arduino Uno is used for motor control and hardware interfacing. The project integrates computer vision and Machine Learning techniques for intelligent decision-making. YOLO (You Only Look Once) is used for object detection, traffic monitoring, and number plate recognition. Ultrasonic sensors help detect obstacles and prevent collisions during navigation. The integration of AI, IoT, and robotics enables efficient real-time operation. The system enhances transportation safety and reduces human intervention. It also supports smart surveillance and monitoring applications. The project demonstrates the practical implementation of automation technologies. It provides hands-on experience in robotics, embedded systems, AI, and computer vision. Overall, the system contributes to the development of intelligent transportation solutions.

Problem Statement :

In many parking areas and roads, vehicles are often parked in no-parking zones, causing traffic congestion and safety problems. Manual monitoring of vehicles is difficult and time-consuming. There is a need for an intelligent autonomous system that can automatically move through an area, detect parked vehicles, identify whether the vehicles are parked legally or illegally, recognize vehicle number plates, classify vehicle models, and send real-time information to a mobile device through WiFi.

This project proposes an AI-based autonomous robotic vehicle using Raspberry Pi, camera modules, sensors, OpenCV, and YOLO algorithms to perform autonomous navigation, vehicle detection, parking analysis, number plate recognition, and mobile monitoring in real time.

An autonomous vehicle project means building a smart vehicle that can:

- move automatically
- detect surroundings
- avoid obstacles
- make decisions using AI
- use sensors and cameras without human control

Your project combines:

- Artificial Intelligence
- Computer Vision
- Raspberry Pi
- IoT
- Mobile Monitoring
- Autonomous Navigation

Objectives of the Project :

The primary objective of this Autonomous Vehicle project is to develop an intelligent and automated vehicle system capable of navigating safely and efficiently with minimal human intervention. The project combines Artificial Intelligence (AI), Robotics, Computer Vision, and embedded systems to create a smart transportation solution.

Main Objectives :

1. Automatic Navigation

To design a vehicle capable of moving from one destination to another automatically by analyzing the surrounding environment and making real-time driving decisions.

2. Obstacle Detection and Avoidance

To detect obstacles such as vehicles, pedestrians, and objects using ultrasonic sensors and camera modules, and safely avoid collisions during movement.

3. Number Plate Recognition

To detect and recognize vehicle number plates using computer vision techniques for traffic monitoring and surveillance applications.

4. Intelligent Traffic Monitoring

To monitor roads and surrounding traffic conditions continuously using camera systems and image processing technologies.

5. Real-Time Decision Making

To enable the autonomous vehicle to analyze sensor data and respond intelligently by controlling speed, direction, and movement automatically.

7. Integration of Hardware and Software

To integrate Raspberry Pi for AI processing and Arduino Uno for motor control and sensor interfacing.

8. Smart Surveillance System

To provide surveillance capabilities through live monitoring and intelligent image analysis for safety and security purposes.

Scope of the Project

- Smart transportation
- Surveillance systems
- Road safety applications

Advantages of the System

- Reduced human errors
- Improved safety
- Intelligent navigation

LITERATURE SURVEY

Existing Research Papers:

Several researchers and technology companies have contributed significantly to the development of autonomous vehicle technologies using Artificial Intelligence, Deep Learning, and Computer Vision. In this project, we referred to research concepts inspired by NVIDIA autonomous driving research and the PilotNet deep learning model for self-driving vehicle navigation.

NVIDIA Autonomous Vehicle Research

NVIDIA introduced advanced AI-based autonomous driving systems that use deep learning and computer vision techniques for real-time vehicle navigation and decision-making. NVIDIA developed intelligent vehicle platforms capable of processing camera data, detecting road conditions, identifying obstacles, and controlling vehicle movement automatically.

Their autonomous driving research mainly focuses on:

- Real-time image processing
- Deep learning-based navigation
- Obstacle detection
- Lane detection
- End-to-end autonomous driving systems
- AI-powered decision making

NVIDIA's AI systems use GPU-based high-performance computing to process large amounts of sensor and camera data quickly and accurately. Their research demonstrated how neural networks can learn driving behavior from real-world road data.

The concepts from NVIDIA research helped us understand:

- Real-time AI processing
- Intelligent vehicle control
- Computer vision implementation
- Autonomous navigation systems

PilotNet Model

PilotNet is a deep learning model developed by NVIDIA for autonomous vehicle steering control. It is an end-to-end Convolutional Neural Network (CNN) designed to predict steering angles directly from camera images without manually programmed driving rules.

The PilotNet system works by:

1. Capturing road images using a front-facing camera
2. Processing images through CNN layers
3. Extracting important road features
4. Predicting steering commands automatically
5. Controlling vehicle movement in real time

Features of PilotNet

- End-to-end deep learning architecture
- Real-time driving predictions
- Automatic feature extraction
- High accuracy in lane following
- Reduced dependency on manual programming

In our project, the concepts of PilotNet inspired:

- Intelligent navigation
- Camera-based road analysis
- Real-time decision-making
- Autonomous steering logic

Relevance to Our Project

The ideas from NVIDIA autonomous driving research and PilotNet helped in designing our smart autonomous vehicle system. We implemented similar concepts such as:

- AI-based image processing
- Real-time object detection using YOLO
- Obstacle avoidance

- Intelligent navigation
- Camera-based monitoring

By studying these research works, we gained knowledge about how modern autonomous vehicles use AI and deep learning to perform safe and efficient transportation tasks.

Limitations of Existing Systems:

- **High cost**

  NVIDIA-based autonomous platforms require powerful processors and expensive hardware components for deep learning computations and real-time image processing.

  Additionally, maintaining and upgrading these intelligent systems also requires considerable financial investment, making them less affordable for small-scale and educational projects.

- **Low accuracy in Complex Environments**

  Although PilotNet and other deep learning models provide good performance under controlled conditions, their accuracy may decrease in complex real-world environments.

- **Dependency on High Computational Power**

  AI-based autonomous systems require large computational resources for processing camera images, sensor data, and deep learning algorithms in real time. This increases power consumption and requires advanced GPUs or AI accelerators for smooth operation.

- **Complex System Integration**

  Integrating sensors, AI models, motor control systems, communication modules, and navigation systems into a single autonomous platform can be technically challenging. System synchronization and hardware compatibility issues may reduce overall efficiency.

SYSTEM ARCHITECTURE

Hardware components:

- **camera**





A camera is one of the most important components in an autonomous vehicle system. It acts as the “eyes” of the vehicle by continuously capturing images and video of the surrounding environment

In this project, the camera module is connected to the Raspberry Pi for image processing and AI-based analysis. The captured images are processed using Computer Vision and Artificial Intelligence algorithms such as YOLO (You Only Look Once) for object detection and monitoring.

Functions of Camera in Autonomous Vehicle

1. Obstacle Detection

The camera detects nearby obstacles such as:

- Vehicles
- Pedestrians
- Roadblocks
- Traffic objects

This helps the autonomous vehicle avoid collisions and move safely.

2. Object Detection and Recognition

Using AI and YOLO algorithms, the camera can identify:

- Cars

- Bikes
- Humans
- Traffic signs
- Road markings

The detected objects are analyzed for intelligent navigation.

3. Number Plate Detection

The camera captures vehicle images and helps in Automatic Number Plate Recognition (ANPR) for surveillance and monitoring purposes.

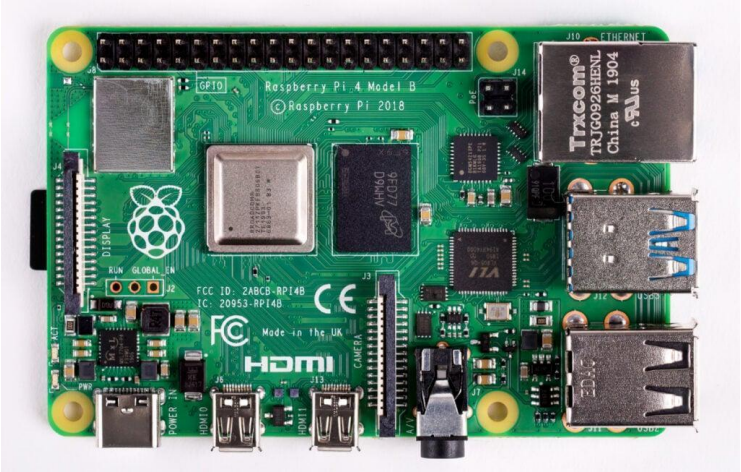
4. Road Monitoring

The camera continuously observes road conditions and traffic environments to support safe navigation and decision-making.

5. Real-Time Surveillance

The system can monitor surroundings continuously and provide live video data for intelligent transportation and security applications.

•Raspberry pi 4 model B



Raspberry Pi 4 Model B is the main processing unit used in this Autonomous Vehicle project. It acts as the brain of the system by performing Artificial Intelligence processing, image analysis, sensor data handling, and communication between different hardware modules.

The Raspberry Pi receives input data from the camera module and sensors, processes the information using AI and Computer Vision algorithms, and sends control commands to the vehicle for intelligent navigation and obstacle avoidance

The Raspberry Pi processes live camera images using Python, OpenCV, and YOLO object detection algorithms. The Raspberry Pi receives input data from the camera and sensors,

processes the information using AI and Computer Vision algorithms, and makes intelligent decisions for autonomous navigation and obstacle avoidance.

•Arduino



In our Autonomous Vehicle project, the Arduino Uno is used as the main hardware control unit responsible for controlling the movement of the vehicle and interfacing with sensors and motor driver modules. It plays an important role in executing real-time movement operations based on commands received from the Raspberry Pi

The Arduino Uno acts as the communication bridge between the intelligent AI processing system and the physical hardware components of the autonomous vehicle. While the Raspberry Pi performs Artificial Intelligence, object detection, and image processing tasks, the Arduino handles low-level motor and sensor control operations efficiently.

Functions of Arduino in the Project

Motor Control

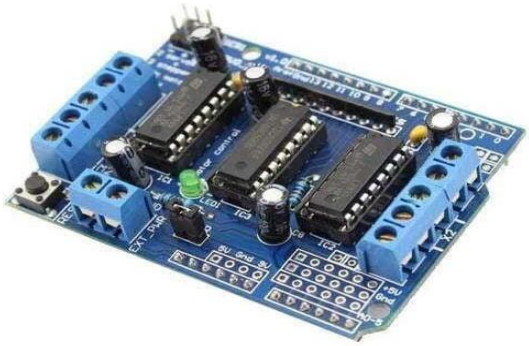
Arduino controls the DC motors through a motor driver module. It manages vehicle movement operations such as:

- Moving forward
- Moving backward
- Turning left
- Turning right
- Stopping the vehicle

Communication with Raspberry Pi

Arduino communicates with the Raspberry Pi using serial communication. The Raspberry Pi processes camera and AI data and sends navigation commands to Arduino for controlling the motors.

Motor Driver



L293D Motor Driver

In this Autonomous Vehicle project, the motor driver module is used to control the speed and direction of the DC motors connected to the vehicle. The motor driver acts as an interface between the Arduino Uno and the motors because the Arduino alone cannot provide sufficient current and voltage required to drive the motors directly.

The motor driver receives control signals from the Arduino and converts them into high-power signals needed for motor operation. It plays an important role in vehicle movement, navigation, and obstacle avoidance.

Functions of Motor Driver in the Project

1. Motor Speed Control

The motor driver controls the speed of the DC motors using PWM (Pulse Width Modulation) signals from Arduino.

2. Direction Control

It helps the vehicle move:

- Forward
- Backward
- Left
- Right
- Stop

by controlling motor rotation direction.

3. Power Amplification

The motor driver supplies sufficient power from the battery to the motors while protecting the Arduino from high current damage.

Power Bank & Lithium-Ion Batteries





In this Autonomous Vehicle project, the power bank is used as the main portable power supply for the Raspberry Pi and other electronic components.

A 20000mAh power bank is used because it provides sufficient backup power for long-duration operation of the Raspberry Pi, camera module, sensors, and communication systems.

Connection of Power Bank to Raspberry Pi

The power bank is connected to the Raspberry Pi using a USB cable through the Raspberry Pi power input port (USB Type-C or Micro USB depending on the model).

Continuous Raspberry Pi Operation

It provides stable voltage and current required for smooth AI processing and image processing operations.

Lithium-Ion Batteries

Lithium-Ion (Li-ion) batteries are rechargeable batteries commonly used in robotics, autonomous vehicles, drones, and electronic devices because of their high energy storage capacity, lightweight design, and long battery life. In this Autonomous Vehicle project, Lithium-Ion batteries are used to provide power to motors, sensors, controllers, and other electronic components.

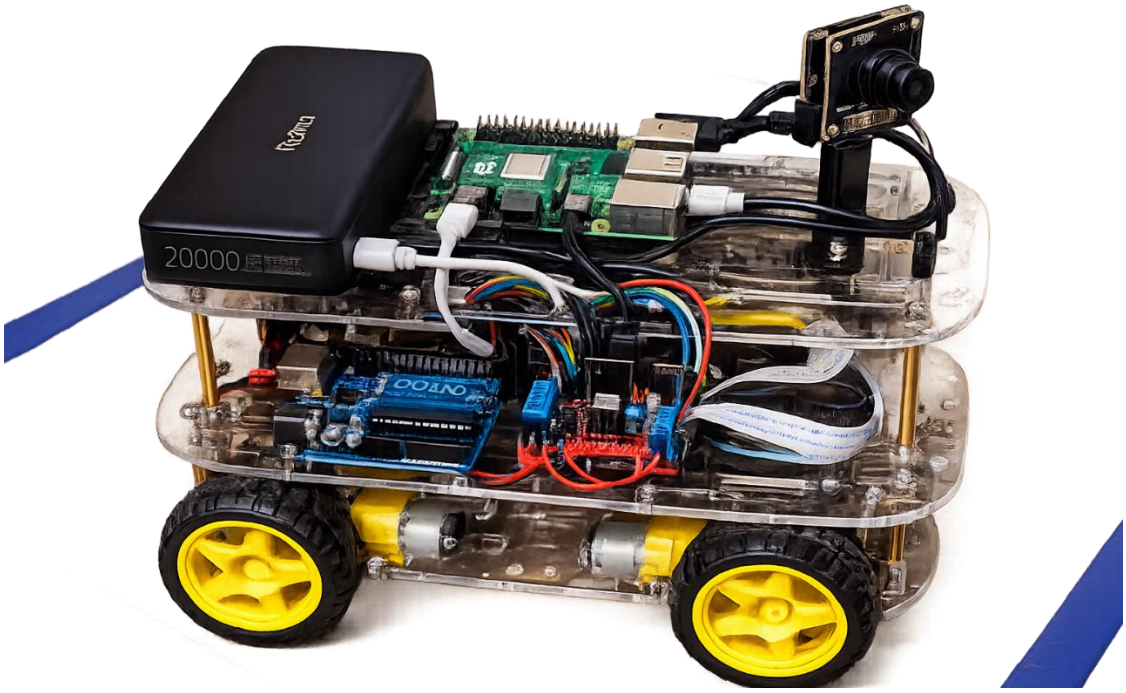
Power Supply for Motors

Lithium-Ion batteries provide sufficient voltage and current to operate DC motors used for vehicle movement.

Portable Autonomous Operation

Since the batteries are rechargeable and lightweight, they allow the vehicle to operate wirelessly without external power connections.

Chassis



In this Autonomous Vehicle project, a double-layer transparent acrylic chassis with 4 wheels and 4 DC motors is used as the base structure of the vehicle. The chassis provides mechanical support for mounting all electronic and hardware components such as the Raspberry Pi, Arduino Uno, motor driver, camera module, power bank, batteries, and connecting cables.

Features of the Chassis

1. Double-Layer Structure

The chassis contains two acrylic layers:

- Upper layer for mounting Raspberry Pi, power bank, and camera
- Lower layer for Arduino, motor driver, batteries, and wiring

This arrangement improves component organization and space management.

2. Transparent Acrylic Material

The transparent acrylic body provides:

- Lightweight structure

- Good durability
- Attractive appearance
- Easy visibility of internal components

3. Four-Wheel Drive System

The chassis is equipped with:

- 4 DC motors
- 4 wheels

This improves:

- Vehicle stability
- Smooth movement
- Better load handling
- Efficient turning capability

4. Component Mounting Support

The chassis supports secure installation of:

- Camera module
- Raspberry Pi
- Arduino Uno
- Motor driver
- Batteries
- Power bank

5. Suitable for Autonomous Navigation

The strong and balanced structure helps the vehicle move safely during obstacle detection, AI processing, and autonomous navigation

Software Requirements :

Python:

Python is a high-level, easy-to-learn programming language widely used in artificial intelligence, machine learning, automation, and computer vision applications. In this autonomous vehicle project, Python was used in the Raspberry Pi for processing camera data, implementing AI algorithms, and controlling the vehicle's intelligent functions.

Python was selected because of its simple syntax, large collection of libraries, and strong support for AI and image processing applications. It helps in real-time decision making and communication between software and hardware components in the autonomous vehicle system.

Python programming language was used in the Raspberry Pi for implementing artificial intelligence, computer vision, and automation tasks in the autonomous vehicle.

Raspberry Pi Software Modules:

pilot_config.py

Purpose

The pilot_config.py file acts as a central configuration module for image preprocessing in the autonomous vehicle system. It ensures that the same image cropping and resizing operations are used during data collection, model training, and real-time driving.

Functions

- Defines image dimensions (200 × 66).
- Stores crop parameters.
- Loads and saves crop settings.
- Performs image cropping and resizing.
- Maintains consistency between training and deployment.

Working

The captured camera image is cropped to remove unnecessary regions such as the sky, walls, and vehicle body. The processed image is resized and supplied to the neural network for training and prediction.

Source Code:

```
============================================================
 pilot_config.py  —  SINGLE SOURCE OF TRUTH
============================================================
 Every file imports the crop + preprocessing from here:
    pilot_collector.py  (shows live crop preview)
    pilot_train.py      (crops training frames)
    pilot_drive.py      (crops inference frames
 Because they all import the SAME crop_and_resize(), the
```

```
 training view and the driving view can never drift apart.
 ____________________________________________________________

 HOW TO SET YOUR CROP (do this once per camera position)
 ____________________________________________________________

  1. Run pilot_collector.py, open the phone UI.
  2. A GREEN BOX shows the crop region live on the camera.
     Everything OUTSIDE the box is darkened.
  3. Drag the 4 sliders (top / bottom / left / right) so the
     box contains ONLY the road + ribbons:
       - cut the ceiling/wall off the TOP
       - cut the Raspberry Pi / cables off the BOTTOM
  4. Tap "SAVE CROP". It writes crop.json next to this file.
  5. Train + drive automatically read crop.json.
 ____________________________________________________________

 DEPLOYING TO THE PI
 ____________________________________________________________

   Always copy crop.json to the Pi together with model.onnx:
      scp model.onnx model.onnx.data crop.json pilot_config.py aak@PI:~/
   If the Pi's crop.json doesn't match the one used in
   training, the model will see a different view and fail.
============================================================
import os, json, numpy as np
RESIZE_W, RESIZE_H = 200, 66


DEFAULT_CROP = {
   "top": 0.18, "bottom": 0.32,
   "left": 0.0, "right": 0.0
}
CROP_FILE = os.path.join(os.path.dirname(__file__), "crop.json")
```

```
def load_crop():
    try:
        with open(CROP_FILE) as f:
            d = json.load(f)
        return {k: float(d.get(k, v)) for k, v in DEFAULT_CROP.items()}
    except:
        return DEFAULT_CROP.copy()
def save_crop(crop):
    c = {k: max(0.0, min(0.8, float(crop.get(k, v))))
         for k, v in DEFAULT_CROP.items()}
    with open(CROP_FILE, "w") as f:
        json.dump(c, f, indent=2)
    return c
def crop_and_resize(img, crop=None):
    crop = crop or load_crop()
    w, h = img.size
    l = int(w * crop["left"])
    r = int(w * (1 - crop["right"]))
    t = int(h * crop["top"])
    b = int(h * (1 - crop["bottom"]))
    if r <= l: r = l + 1
    if b <= t: b = t + 1
    img = img.crop((l, t, r, b)).resize((RESIZE_W, RESIZE_H))
    return np.asarray(img, dtype=np.float32) / 255.0

if __name__ == "__main__":
    print("Crop:", load_crop())
    print(f"Input Size: {RESIZE_W}x{RESIZE_H}")
    print("Crop File:", CROP_FILE)
```

pilot_collector.py

Purpose

The pilot_collector.py module is responsible for collecting training data from the autonomous vehicle.

Functions

- Captures camera images.
- Receives steering and speed commands.
- Controls Arduino motors.
- Stores images and driving parameters.
- Generates datasets for model training.

Working

The Raspberry Pi camera continuously captures images while the user manually drives the vehicle. Each image is stored along with steering angle, speed, and motor PWM values. These records are saved in CSV format and later used for neural network training.

Input

- Camera Frames
- Steering Commands
- Speed Commands

Output

- Image Dataset
- Log CSV File

Source Code:

```
import os, csv, time, datetime, threading, serial
from flask import Flask, request, jsonify
from picamera2 import Picamera2
app = Flask(__name__)
DATASET = os.path.expanduser("~/dataset")
```

```
os.makedirs(DATASET, exist_ok=True)
arduino = serial.Serial('/dev/ttyUSB0', 9600, timeout=1)
picam = Picamera2()
picam.configure(picam.create_video_configuration(
  main={"size": (320,240)}
))
picam.start()
state = {"angle":0, "speed":0, "recording":False, "count":0}
def create_session():
  ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
  session = os.path.join(DATASET, f"session_{ts}")
  frames = os.path.join(session, "frames")
  os.makedirs(frames, exist_ok=True)
  csvfile = open(os.path.join(session,"log.csv"),"w",newline="")
  writer = csv.writer(csvfile)
  writer.writerow(["image","angle","speed"])
  return frames, writer
frames, writer = create_session()

@app.route("/control", methods=["POST"])
def control():
  data = request.json
  state["angle"] = int(data.get("angle",0))
  state["speed"] = int(data.get("speed",0))
  arduino.write(
    f"{state['angle']},{state['speed']}\n".encode()
  )
  return jsonify({"status":"ok"})
def logger():
```

```
    while True:
        if state["recording"]:
            name = f"{state['count']:06d}.jpg"
            picam.capture_file(
                os.path.join(frames,name)
            )
            writer.writerow([
                name,state["angle"],state["speed"]
            ])
            state["count"] += 1
        time.sleep(0.05)


if __name__ == "__main__":
    state["recording"] = True
    threading.Thread(target=logger, daemon=True).start()
    app.run(host="0.0.0.0", port=5000)
```

pilot_model.py

Purpose

The pilot_model.py file implements the PilotNet deep learning architecture used for autonomous steering prediction.

Functions

- Defines convolutional neural network layers.
- Extracts road features from images.
- Predicts steering angle.
- Generates vehicle navigation commands.

Working

The network receives processed camera images as input. Convolution layers extract road features and fully connected layers determine the steering direction. The final output is a steering value ranging from -1 to +1.

Input

- RGB Camera Image

Output

- Steering Angle

Source code:

```
import torch, pandas as pd
from pathlib import Path
from PIL import Image
from torch.utils.data import Dataset, DataLoader, random_split
from pilot_model import PilotNet
from pilot_config import crop_and_resize
class PilotDataset(Dataset):
  def __init__(self, root):
    self.samples = []
    for s in Path(root).glob("session_*"):
      df = pd.read_csv(s/"log.csv")
      for _, r in df.iterrows():
        f = s/"frames"/str(r["frame_file"])
        if f.exists():
          self.samples.append((f, float(r["angle"])))
  def __len__(self): return len(self.samples)
  def __getitem__(self, i):
    f, angle = self.samples[i]
    img = Image.open(f).convert("RGB")
    img = torch.tensor(crop_and_resize(img)).permute(2,0,1).float()
    return img, torch.tensor([angle/100], dtype=torch.float32)
```

```
ds = PilotDataset("./dataset")
train_ds, val_ds = random_split(ds,[int(0.8*len(ds)),len(ds)-int(0.8*len(ds))])
train_loader = DataLoader(train_ds,batch_size=32,shuffle=True)
val_loader = DataLoader(val_ds,batch_size=32)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = PilotNet().to(device)
loss_fn = torch.nn.SmoothL1Loss()
opt = torch.optim.Adam(model.parameters(),lr=1e-3)
for epoch in range(10):
  model.train()
  train_loss = 0
  for x,y in train_loader:
    x,y = x.to(device),y.to(device)
    loss = loss_fn(model(x)["steering"],y)
    opt.zero_grad(); loss.backward(); opt.step()
    train_loss += loss.item()
  model.eval()
  val_loss = 0
  with torch.no_grad():
    for x,y in val_loader:
      x,y = x.to(device),y.to(device)
      val_loss += loss_fn(model(x)["steering"],y).item()
  print(f"Epoch {epoch+1}: "
      f"Train={train_loss/len(train_loader):.4f} "
      f"Val={val_loss/len(val_loader):.4f}")
torch.save(model.state_dict(),"model.pt")
print("Model saved!")
```

pilot_train.py

Purpose

The pilot_train.py module trains the PilotNet model using the collected driving dataset.

Functions

- Loads image dataset.
- Removes unwanted samples.
- Applies image augmentation.
- Performs neural network training.
- Saves trained model files.

Working

The training module reads all collected images and driving records. Data preprocessing techniques such as image flipping and smoothing are applied. The PilotNet architecture is trained using the processed dataset. After successful training, the model is exported into ONNX format for deployment on Raspberry Pi.

Input

- Dataset Images
- Steering Data

Output

- model.pt
- model.onnx

source code:

```
import torch, pandas as pd
from pathlib import Path
from PIL import Image
from torch.utils.data import Dataset, DataLoader
from pilot_model import PilotNet
from pilot_config import crop_and_resize
```

```
class PilotDataset(Dataset):
    def __init__(self, root):
        self.samples = []
        for s in Path(root).glob("session_*"):
            csv = s / "log.csv"
            frames = s / "frames"
            if csv.exists():
                df = pd.read_csv(csv)
                for _, r in df.iterrows():
                    f = frames / str(r["frame_file"])
                    if f.exists():
                        self.samples.append((f, float(r["angle"])))

    def __len__(self):
        return len(self.samples)
    def __getitem__(self, i):
        f, angle = self.samples[i]
        img = Image.open(f).convert("RGB")
        img = crop_and_resize(img)
        img = torch.tensor(img).permute(2,0,1).float()
        steer = torch.tensor([angle/100], dtype=torch.float32)
        return img, steer
ds = PilotDataset("./dataset")
loader = DataLoader(ds, batch_size=32, shuffle=True)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = PilotNet().to(device)
loss_fn = torch.nn.SmoothL1Loss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
for epoch in range(10):
    total = 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        pred = model(x)["steering"]
        loss = loss_fn(pred, y)
        opt.zero_grad()
        loss.backward()
        opt.step()
        total += loss.item()
    print(f"Epoch {epoch+1}: {total/len(loader):.4f}")
torch.save(model.state_dict(), "model.pt")
dummy = torch.zeros(1,3,66,200).to(device)
torch.onnx.export(model, dummy, "model.onnx",
                  input_names=["frame"],
                  output_names=["steering"])
print("Training complete!")
```

• **Raspberry Pi Side**	
• **File**	• **Purpose**
• pilot_config.py	• Image preprocessing and crop settings
• pilot_collector.py	• Collects camera images and driving data
• pilot_model.py	• Defines the PilotNet neural network architecture
• pilot_train.py	• Trains the AI model using collected data

Software Modules Implemented on Raspberry Pi

1. pilot_config.py
2. pilot_collector.py
3. pilot_model.py

4. pilot_train.py

Hardware Interaction

- Raspberry Pi 4 — Image Processing and AI Computation
- Pi Camera — Image Acquisition
- Arduino Uno — Motor Control
- DC Motors — Vehicle Movement

Arduino Motor Control Module:

File: AI Car Controller Firmware (Arduino Code)

```
import torch, pandas as pd
from pathlib import Path
from PIL import Image
from torch.utils.data import Dataset, DataLoader
from pilot_model import PilotNet
from pilot_config import crop_and_resize

class PilotDataset(Dataset):
  def __init__(self, root):
    self.samples = []
    for s in Path(root).glob("session_*"):
      csv = s / "log.csv"
      frames = s / "frames"
      if csv.exists():
        df = pd.read_csv(csv)
        for _, r in df.iterrows():
          f = frames / str(r["frame_file"])
          if f.exists():
            self.samples.append((f, float(r["angle"])))
```

```
    def __len__(self):
        return len(self.samples)
    def __getitem__(self, i):
        f, angle = self.samples[i]
        img = Image.open(f).convert("RGB")
        img = crop_and_resize(img)
        img = torch.tensor(img).permute(2,0,1).float()
        steer = torch.tensor([angle/100], dtype=torch.float32)
        return img, steer
ds = PilotDataset("./dataset")
loader = DataLoader(ds, batch_size=32, shuffle=True)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = PilotNet().to(device)
loss_fn = torch.nn.SmoothL1Loss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(10):
    total = 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        pred = model(x)["steering"]
        loss = loss_fn(pred, y)
        opt.zero_grad()
        loss.backward()
        opt.step()
        total += loss.item()
    print(f"Epoch {epoch+1}: {total/len(loader):.4f}")
torch.save(model.state_dict(), "model.pt")
dummy = torch.zeros(1,3,66,200).to(device)
torch.onnx.export(model, dummy, "model.onnx",
```

```
        input_names=["frame"],
        output_names=["steering"]
print("Training complete!")
```

Methodology

Step 1: Data Collection

Using pilot_collector.py, images and steering data are collected while manually driving the vehicle.

Step 2: Data Preprocessing

Using pilot_config.py, images are cropped and resized to remove unwanted regions and maintain consistency.

Step 3: Model Development

Using pilot_model.py, the PilotNet convolutional neural network architecture is created.

Step 4: Model Training

Using pilot_train.py, the collected dataset is used to train the PilotNet model.

Step 5: Model Deployment

The trained ONNX model is deployed on Raspberry Pi for autonomous navigation.

Step 6: Autonomous Driving

The vehicle captures live images, predicts steering angles, and navigates automatically.

References

Books

- Autonomous driving – author markus maurer,j.christain gerdes
- Computer vision-algorithms and applications

Research Papers

- You Only Look Once: Unified, Real-Time Object Detection
  Authors: Joseph Redmon et al.
- End to End Learning for Self-Driving Cars
  Authors: Researchers from NVIDIA

Websites

Raspberry Pi Documentation
Raspberry Pi setup and programming guides.

Arduino Documentation

Motor control and sensor interfacing.

TensorFlow Documentation

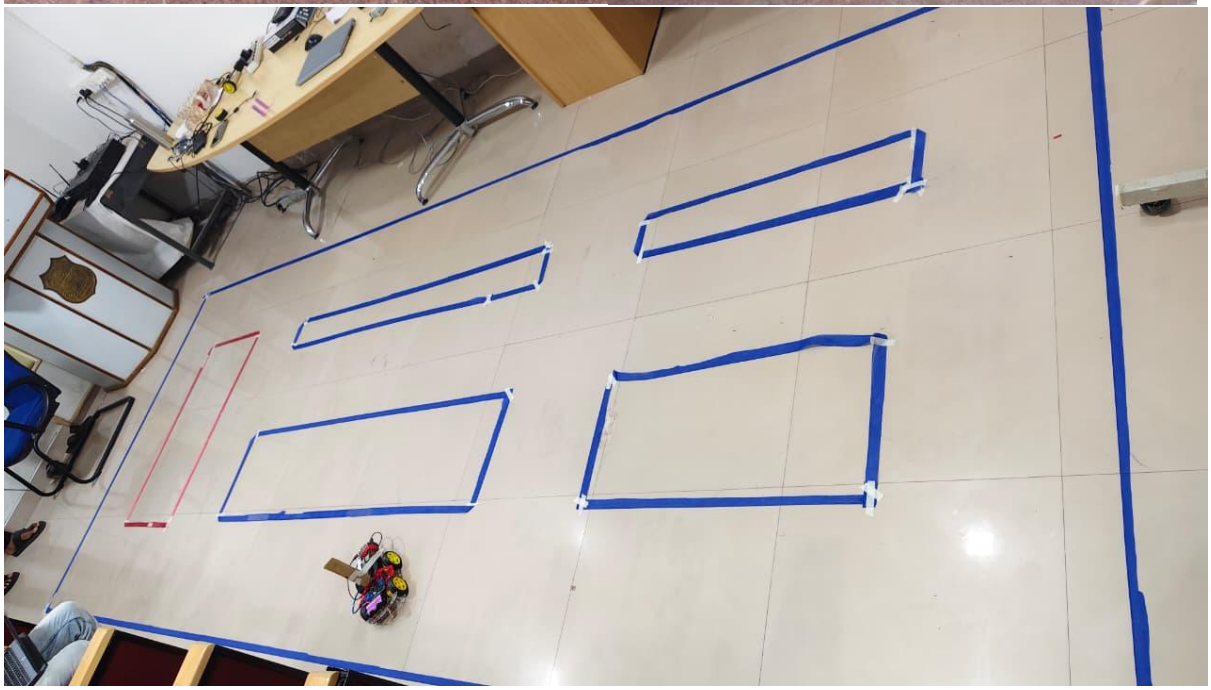
Machine learning and deep learning resources.

Project Photographs

Guidance provided by Dr.L.Anjaneyulu for autonomous vehicle project







Final Prototype Vehicle

Prototype Vehicle Overview

The final prototype vehicle was developed by transforming a Toyzone Buster electric ride-on vehicle into an intelligent autonomous platform capable of performing real-time perception, decision-making, and autonomous navigation. The vehicle serves as a compact experimental platform for validating artificial intelligence, and autonomous driving technologies in a safe and controlled environment.

Multiple front-mounted cameras continuously capture real-time visual data, which is processed using advanced computer vision and deep learning algorithms to detect road lanes, obstacles

The original electrical and mechanical architecture of the Toyzone Buster vehicle was carefully modified to accommodate additional computing hardware, motor drivers, sensors, power management modules, and communication interfaces without affecting the structural stability of the vehicle. Dedicated mounting brackets were designed to securely install cameras, processing units, and supporting electronics while maintaining proper weight distribution and accessibility for maintenance

A laptop was integrated into the development platform for monitoring system performance, executing AI models, visualizing camera feeds, collecting experimental data, and performing software debugging during testing. The complete system was evaluated under various operating conditions to verify the reliability of perception, navigation, and vehicle control.

Vehicle Specifications

The autonomous prototype was developed using a Toyzone Buster electric ride-on vehicle as the base platform. The vehicle was upgraded by integrating artificial intelligence, embedded control systems, computer vision hardware, and autonomous navigation components while preserving the original electric drive mechanism. The final prototype specifications are presented in Table

Final Prototype Vehicle Specifications

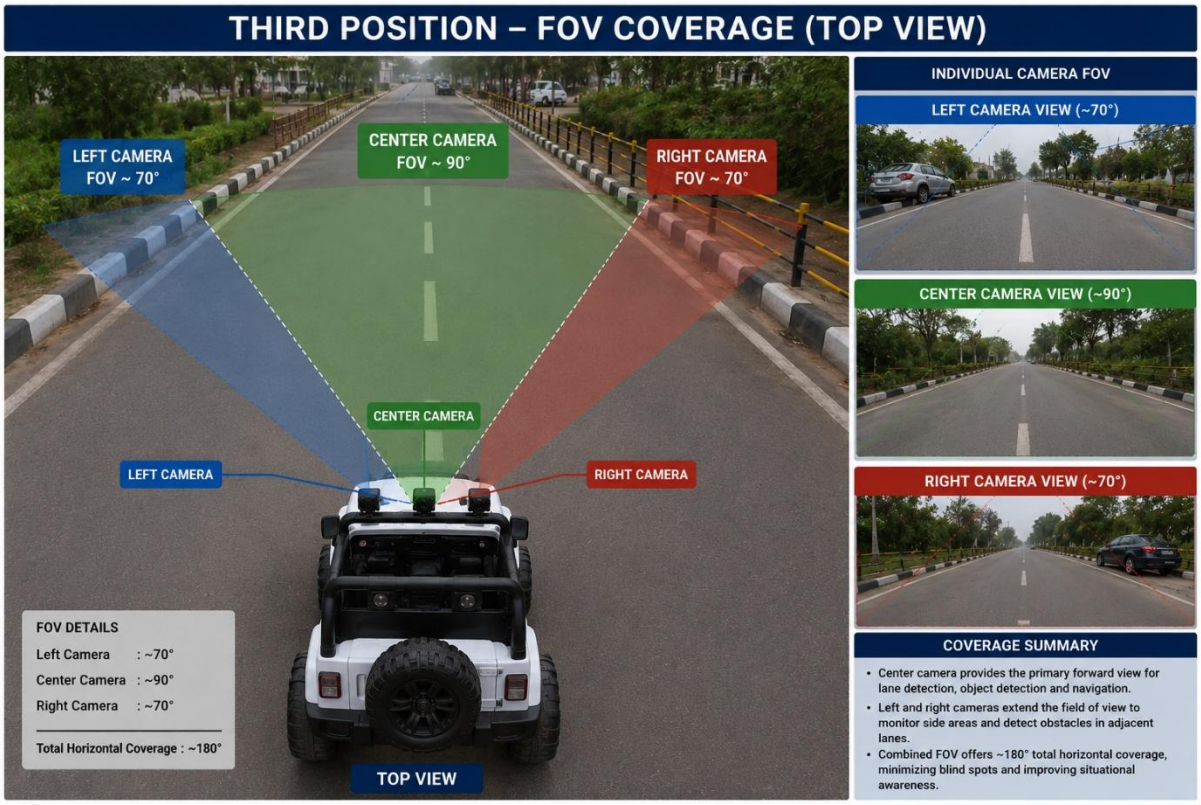
Parameter	Specification
Prototype Platform	Toyzone Buster Electric Ride-On Vehicle
Vehicle Type	AI-Based Autonomous Electric Vehicle Prototype
Power Source	12 V DC Rechargeable Battery
Battery Capacity	8 Ah
Drive System	Rear Wheel Drive (Single DC Motor)
Steering System	Electric Steering Motor
Processing Unit	Raspberry Pi 4 Model B
Motor Controller	Arduino Uno
Vision System	Three Front-Mounted USB Web Cameras
AI Algorithms	YOLO Object Detection, Lane Detection.
Obstacle Detection	Ultrasonic Sensors
Programming Languages	Python, C/C++
Vehicle Control	Autonomous Steering and Speed Control
Monitoring Interface	External Laptop for AI Processing and System Monitoring
Prototype Application	Autonomous Navigation, Object Detection, Research and Educational Demonstration

Camera Mounting and Placement

The vision system of the autonomous vehicle consists of three high-resolution USB cameras strategically mounted on the front section of the Toyzone Buster electric vehicle to provide comprehensive environmental perception. The camera arrangement was designed to maximize the field of view while minimizing blind spots, enabling reliable real-time image acquisition for autonomous navigation

The cameras were securely mounted using custom-designed mounting brackets fixed to the front body of the vehicle. The mounting structure was engineered to minimize vibrations and maintain camera stability during vehicle movement, ensuring consistent image quality for computer vision processing. The camera cables were routed through protective cable management channels to prevent interference with moving components and to improve the overall reliability and appearance of the prototype.

This multi-camera configuration forms a critical component of the autonomous driving system and contributes to safer and more reliable vehicle operation in dynamic environments.



Electrical Wiring Layout

The autonomous vehicle retains the **original electrical system of the Toyzone Buster ride-on car** without any internal modifications. Instead of altering the vehicle's wiring or control circuitry, the autonomous control system is implemented by modifying the **wireless remote controller**. Transistors were integrated into the remote control circuit to electronically simulate button presses for forward, reverse, left, and right movements.

The laptop receives real-time video from the mounted USB cameras and processes the images using OpenCV and YOLO-based algorithms. Based on the detected lane and obstacle information, in laptop executes Python code that activates the appropriate transistors connected to the remote controller. The remote then wirelessly transmits the control commands to the Toyzone Buster vehicle, enabling autonomous navigation while preserving the original vehicle hardware.

Testing Environment :

The autonomous vehicle was tested in a **controlled indoor environment** to evaluate its navigation and object detection capabilities under safe and repeatable conditions. A predefined track was created using marked lanes, and different obstacles were placed at various locations to simulate real-world driving scenarios. The three front-mounted USB cameras continuously captured the surrounding environment

The autonomous vehicle was tested on a **wide, tree-lined campus roadway** that closely resembles a real-world urban driving environment. The testing location provided a smooth asphalt surface with clearly visible lane markings, allowing the vehicle to perform autonomous navigation under realistic conditions. The road was surrounded by trees, pedestrian pathways, and street lighting, creating a natural outdoor environment suitable for evaluating lane-following performance and vehicle stability.

The testing environment was selected to simulate practical driving conditions while ensuring safe operation of the prototype. Multiple test runs were conducted to evaluate lane tracking accuracy, steering response, and the overall reliability of the autonomous control system.



Performance Evaluation

The performance of the autonomous Toyzone Buster vehicle was evaluated through multiple test runs in an outdoor campus environment. The evaluation focused on the vehicle's ability to follow lane markings, maintain stable navigation, and respond to changes in the road using real-time image processing.

Although minor variations in steering accuracy were observed under changing lighting conditions and faded lane markings, the overall system operated reliably and achieved the intended objectives of autonomous navigation.

During testing, the vehicle successfully maintained its path on straight road sections and executed gradual turns by following the lane markings.

Final Prototype Images

The following images present the final prototype of the autonomous vehicle developed during the internship at the **NIT Warangal**. The prototype was successfully integrated onto a **Toyzone car vehicle**, where the original remote control was modified using electronic circuitry and software to enable autonomous operation.

The images also show the project team with the completed prototype and the project mentor during the successful completion of the internship.






.V.A. SASTRY CENTRE
INNOVATION AND DEVELOPMENT




DR.V.A. SASTRY CE
FOR INNOVATION AND DEVELO



Result

The autonomous vehicle project was implemented in two phases to ensure reliable system development and validation. In the first phase, a small-scale Toyzone car chassis prototype was developed to verify the complete control pipeline. A single camera was used to capture driving images, while an Arduino Uno received steering commands generated through a joystick connected via a transistor-based switching circuit. This prototype enabled verification of image acquisition, dataset collection, steering data labeling, and motor response before moving to the full-scale vehicle.

In the second phase, the complete autonomous vehicle was developed using the same joystick–transistor–Arduino Uno control architecture. Three cameras were mounted at the front of the vehicle following the NVIDIA PilotNet approach, providing left, center, and right viewpoints. The multi-camera configuration generated a richer training dataset and improved the vehicle's ability to recover from slight deviations from the center of the driving path. A laptop was used as the onboard computing platform for both model training and real-time inference.

A convolutional neural network based on the NVIDIA PilotNet architecture was implemented using PyTorch. The network accepted preprocessed camera images as input and directly predicted steering angles. The model was trained using Mean Squared Error (MSE) loss with the PyTorch Dataset and DataLoader framework. During real-time operation, the trained model continuously predicted steering commands, which were transmitted to the Arduino Uno to control the vehicle's motors.

Experimental testing demonstrated that the autonomous vehicle successfully navigated the predefined training route without manual joystick intervention. The vehicle accurately followed the learned path and reached the intended destination, validating the effectiveness of the end-to-end deep learning approach for autonomous steering.

Key Takeaways

- The transistor-based interface provided a simple, reliable, and cost-effective method for converting joystick signals into Arduino-compatible control inputs.

- A laptop effectively served as the onboard computing platform, eliminating the immediate need for dedicated embedded hardware during development.
- The three-camera configuration based on the NVIDIA PilotNet methodology significantly improved dataset diversity and steering stability compared with the single-camera prototype.
- Developing and validating the small-scale prototype reduced implementation risks and simplified debugging before deployment on the complete vehicle.
- The trained model demonstrated reliable performance on the trained route; however, its capability on unseen environments and routes requires further investigation.

Future Scope

- Deploy the trained deep learning model on an embedded AI platform such as the NVIDIA Jetson Orin Nano to achieve a compact, fully autonomous onboard system.
- Expand the training dataset by collecting data from multiple routes, varying road conditions, lighting environments, and weather conditions to improve model generalization.
- Integrate real-time object detection using YOLO to enable obstacle detection and collision avoidance in addition to autonomous path following.
- Incorporate GPS and waypoint-based navigation for autonomous travel across multiple destinations instead of a single predefined route.
- Apply advanced data augmentation techniques, including brightness adjustment, shadow simulation, image rotation, and horizontal flipping with steering angle correction, to improve robustness.
- Perform quantitative performance evaluation using metrics such as steering angle error, trajectory deviation, and path-following accuracy alongside qualitative testing.